Окно контекста, промпт и лестница памяти

ИИ-ассистенты для исследователя. AI4Science — неделя 3 из 12

Окно контекста, промпт и лестница памяти

А. П. Халов, О. М. Атаева

ФИЦ ИУ РАН МФТИ ШАД Яндекса

Осень 2026



План недели

Агент в коде

Архитектура: откуда предел окна

Предел окна

Окно агента наполняется само

Промпт и примеры

Сжатие и память

Спецификации проекта

Подагенты

Практика и итоги

Начало занятия — ноутбук «Агент в коде»: цикл агента целиком.

Основная часть идёт снизу вверх: от устройства модели к тому, что исследователь пишет в своём проекте. Каждый уровень работает в границах нижнего, поэтому рекомендации верхних уровней — писать коротко, класть важное в начало и в конец, выносить лишнее из окна — выводятся из нижних.

В конце — практика недели и постановка ДЗ 1.

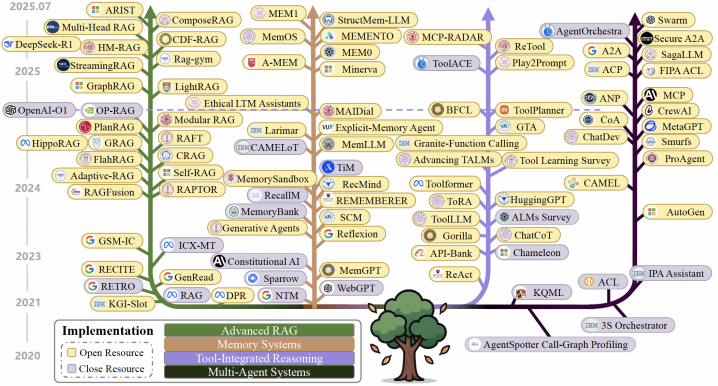
Ноутбук «Агент в коде»: что внутри

Агент собран из нескольких функций на Python; «проект» — словарь файлов в памяти, доступа к диску нет.

Понятие	**В ноутбуке**	**Что это**
окно s_t	`context`	список сообщений: системный промпт, запрос, вызовы, наблюдения
среда e_t	`env`	словарь «имя файла $\to$ текст»
действия $\mathcal{A}$	`TOOLS`	пять функций и их описания для модели
политика π	`ask`	один вызов модели: окно $\to$ сообщение
шаг	`step`	порождение, исполнение, наблюдение, запись в окно
ведущее приложение	`run`	цикл, критерий готовности, защита файлов, остановка при повторах
критерий готовности φ	`phi(env)`	функция, написанная до запуска

Ноутбук: `agent_notebook/agent_loop.ipynb` в общей папке курса. Ключ инференса берётся из настроек учебного агента.

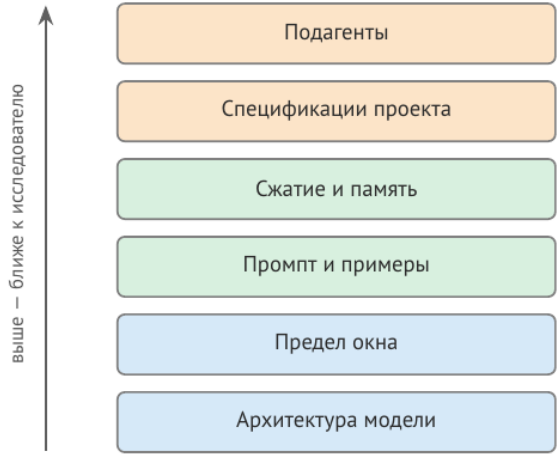
Инженерия контекста: карта области



Контекст — результат сборки $C = A(c_{\text{instr}}, c_{\text{know}}, c_{\text{tools}}, c_{\text{mem}}, c_{\text{state}}, c_{\text{query}})$: инструкции, знания, инструменты, память, состояние, запрос.

Источник рисунка: [112] L. Mei и др., «A survey of context engineering for large language models», 2025 — рис. 2, ур. 1 (обзор более 1400 работ; не рецензируемое).

Уровни управления контекстом



каждый уровень работает в границах нижнего

Токен — единица текста при генерации

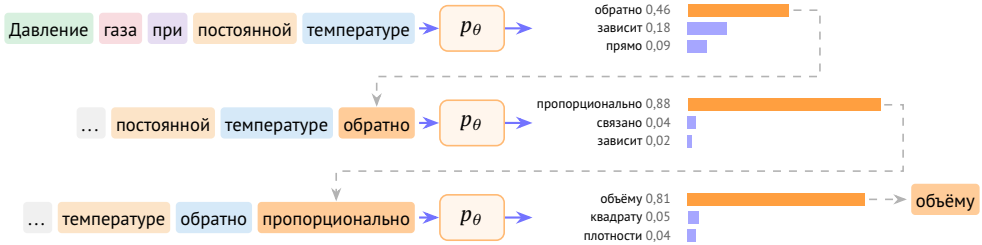
Текст разбивается на элементы словаря $\mathcal{V}$; частые сочетания объединены в один элемент (BPE) [26]. Рассмотрим разбиение токенизатором моделей Qwen3 [40], $|\mathcal{V}| = 151\,669$.

текст	токены и их номера в словаре	токенов
английский	pressure (45974) · ␣is (374) · ␣invers (64583) · ely (974) · ␣proportional (54172) · ␣to (311) · ␣volume (8123)	7
русский	дав · ление · ␣обрат · но · ␣пр · оп · ор · ци · она · ль · но · ␣объ · ё · му	14
формула	p · ␣* · ␣V · ␣== · ␣const	5
Lean	theorem · ␣foo · ␣: · ␣a · ␣+ · ␣b · ␣= · ␣b · ␣+ · ␣a · ␣:= · ␣by · ␣ring	13

- На вход модели подаётся последовательность $x_1, \ldots, x_N \in \mathcal{V}$.
  Естественный язык, формулы и код кодируются одним словарём.
- Окно контекста, лимиты и стоимость измеряются в токенах.
  Русская фраза примера заняла 14 токенов, английская того же смысла — 7.

Знак ␣ — пробел в начале токена. Разбиение вычислено токенизатором модели Qwen3-0.6B.

Авторегрессионное порождение текста

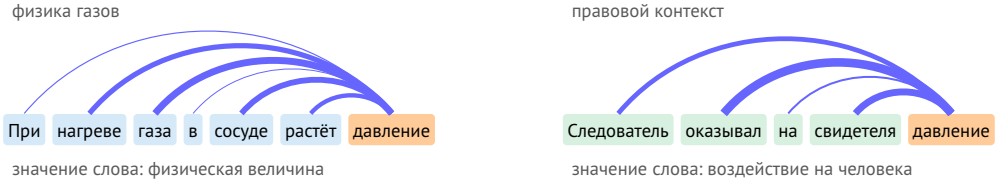


$$p_\theta(x_1, \ldots, x_N) \;=\; \prod_{j=0}^{N-1} p_\theta\left(x_{j+1} \mid x_{1:j}\right)$$

- На каждом шаге модель выдаёт распределение над $\mathcal{V}$; выбранный токен дописывается к входу, и шаг повторяется до служебного токена конца [27], [30].
- Неверное продолжение имеет ненулевую вероятность.

Для наглядности токены показаны словами; вероятности условные.

Контекст и внимание



Упрощённо (без линейных проекций) новое представление токена j — взвешенная сумма представлений токенов префикса:

$$h'_j = \sum_{i \le j} \alpha_{ji}\, h_i, \qquad \alpha_{ji} \ge 0, \quad \sum_{i \le j} \alpha_{ji} = 1.$$

- Веса α_{ji} вычисляются по самим представлениям, поэтому один и тот же токен в разных контекстах получает разные h'_j [28], [29], [38].
- Трансформер — десятки слоёв внимания и преобразований.

Толщина линии — вес α_{ji} последнего токена; значения условные.

Внимание: запрос, ключ, значение

Каждый токен j получает три вектора, вычисленные обученными матрицами из его представления h_j:

$$q_j = W_Q h_j \ \text{(запрос)}, \qquad k_j = W_K h_j \ \text{(ключ)}, \qquad v_j = W_V h_j \ \text{(значение)}.$$

1. **Оценка:** насколько токен i подходит запросу токена j — скалярное произведение $q_j \cdot k_i / \sqrt{d_k}$.
2. **Веса:** оценки по всем $i \le j$ нормируются softmax в веса $\alpha_{ji} \ge 0$ с суммой 1.
3. **Результат:** новое представление — взвешенная сумма значений $h'_j = \sum_{i \le j} \alpha_{ji} v_i$.

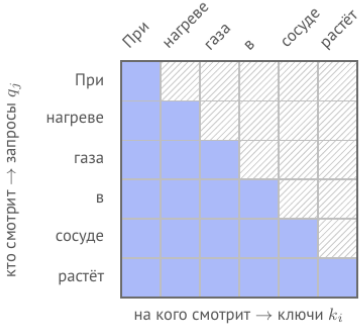
Для всех токенов сразу, в матрицах Q, K, V размера $N \times d_k$:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V.$$

---

[28] A. Vaswani и др., «Attention is all you need», NeurIPS 2017, §3.2, ур. 1. Слова «запрос», «ключ», «значение» — названия векторов, не их смысл. V здесь — матрица значений (обозначение статьи).

Матрица внимания и причинная маска



нижний треугольник считается:
токен j смотрит на себя и на прошлые $i \le j$

верхний треугольник закрыт маской:
будущих токенов ещё нет, их вес $= 0$

каждая строка — распределение:
веса $\alpha_{ji} \ge 0$, их сумма $= 1$

клеток: $N(N+1)/2$ на голову и слой

Модель порождает текст слева направо, поэтому токен видит только прошлое: в матрице оценок $QK^\top$ клетки выше диагонали закрыты. Отсюда нижний треугольник — его и нужно считать.

[28] A. Vaswani и др., «Attention is all you need», NeurIPS 2017, §3.2.3 (маска в декодере).

Стоимость внимания

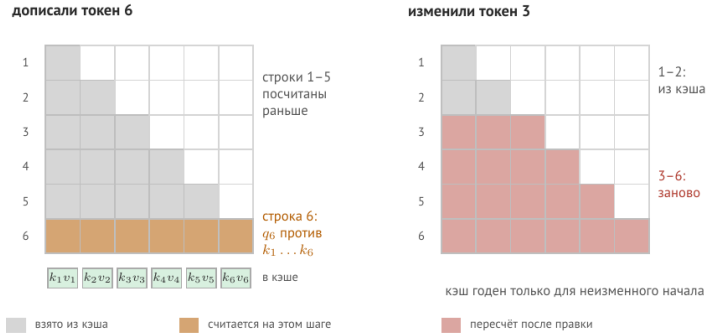
- Число пар токенов в треугольнике — $N(N+1)/2$, на каждую пару — скалярное произведение длины d_k. Операций на голову и слой — $O(N^2 d_k)$; у рекуррентного слоя — $O(Nd^2)$ [28, табл. 1].
- При длинном окне ($N \gg d$) стоимость определяет квадрат длины. Например, $N = 262\,144$: около $3{,}4 \cdot 10^{10}$ пар на каждую голову каждого слоя.
- Промежуточная матрица оценок $N \times N$ при прямом вычислении занимает память $O(N^2)$.

Длина окна — главный множитель стоимости и растет квадратично.

---

[28] A. Vaswani и др., «Attention is all you need», NeurIPS 2017, табл. 1. В статье длина обозначена n; в курсе n — число шагов агента, длина окна — N.

Зачем платить больше если есть KV-кэш?



- Из-за маски строки прошлых токенов не зависят от новых: их ключи и значения можно посчитать один раз и потом переиспользовать. Новый токен добавляет одну строку: q_N.
- Любое изменение в середине меняет все строки после неё: из кэша берётся только неизменное начало последовательности.

[99] W. Kwon и др., «Efficient memory management for large language model serving with PagedAttention», SOSP 2023, §2–3. [86] N. Shazeer, «Fast transformer decoding», 2019, §2.4.

Сколько стоит KV-кэш

Qwen3 235B (версия 2507): 94 слоя, 64 головы запросов и 4 головы ключей и значений (т.н. групповое внимание), размер головы 128, числа в bfloat16 (2 байта) [119].

$$2 \cdot 94 \cdot 4 \cdot 128 \cdot 2 \text{ байта} = 188 \text{ КВ на токен.}$$

Длина окна	**KV-кэш одного запроса**	**если бы у каждой из 64 голов свои K/V**
4 096	0,7 GB	12 GB
32 768	5,9 GB	94 GB
262 144	47 GB	752 GB

Провайдер может хранить кэш в меньшей точности. Общие для групп голов ключи и значения уменьшают кэш в 16 раз.

[119] Qwen Team, «Qwen3-235B-A22B-Instruct-2507», карточка модели и `config.json`, Hugging Face, 2025.

Как уменьшают KV-кэш

Приём	**Что делает**	**Эффект**
общие ключи и значения (MQA)	все головы внимания делят один набор K/V	кэш меньше в h раз;[86]
групповое внимание (GQA)	G групп голов делят K/V	промеж. вар-т [94]
латентное внимание (MLA)	кэшируется сжатый вектор вместо K/V	кэш $-93{,}3\,\%$ [106]
вытеснение из кэша	хранятся «тяжёлые» и последние токены	- 20 % без потерь [96]

[86] N. Shazeer, «Fast transformer decoding: One write-head is all you need», 2019. [94] J. Ainslie и др., «GQA», EMNLP 2023. [106] DeepSeek-AI, «DeepSeek-V2», 2024 (не рецензируемое). [96] Z. Zhang и др., «H2O: Heavy-hitter oracle», NeurIPS 2023.

Окно контекста

Окно контекста s_t — вся последовательность токенов, которая подаётся модели на шаге t: системный промпт, AGENTS.md, описания инструментов, запросы, вывод инструментов, рассуждения. Длина окна $N = |s_t|$ ограничена пределом $N_{\max}$.

Модель учебного агента	$N_{\max}$, **токенов**
Qwen3 235B, Qwen3.6 35B	262 144
DeepSeek V4 Flash	1 048 576
gpt-oss-120b, gpt-oss-20b	131 072

Что вне окна контекста — для агента не существует.

[49] Yandex Cloud, «Available generative models», документация Yandex AI Studio (обращение 14.09.2026).

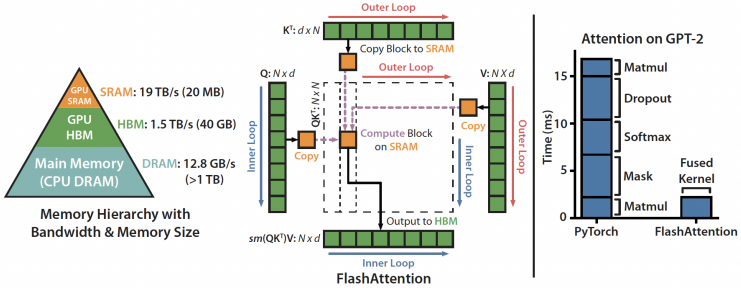
Почему у окна есть предел



- **Обучение.** Модель видела последовательности до определённой длины; поведение на незнакомых позициях не гарантировано.
- **Растяжение.** Позиции пересчитывают и дообучают модель на длинных текстах: YaRN — до 64K–128K при дообучении на менее чем 0,1 % данных предобучения [98]. У Qwen3 235B версии 2507 — 262 144 позиции [119].
- **Память.** KV-кэш полного окна — десятки гигабайт на один запрос.
- **Проверка.** Заявленная длина — верхняя граница: Yarn-Llama-2-7B заявлена на 128K, эффективная длина — меньше 4K [105, табл. 12].

[98] B. Peng и др., «YaRN», ICLR 2024. [119] Qwen Team, карточка модели, 2025. [105] C.-P. Hsieh и др., «RULER», COLM 2024.

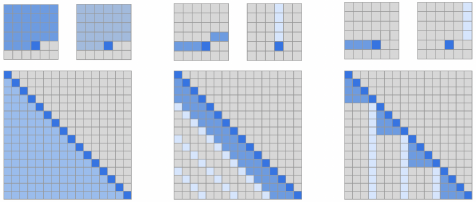
FlashAttention: точное внимание блоками



- Матрица $N \times N$ считается блоками в быстрой памяти ускорителя и целиком не записывается. Операций по-прежнему $O(N^2 d)$; дополнительная память — $O(N)$.
- Ускорение даёт меньший обмен с медленной памятью: внимание в GPT-2 быстрее в 7,6 раза.

Источник рисунка: [93] T. Dao и др., «FlashAttention: Fast and memory-efficient exact attention with IO-awareness», NeurIPS 2022, рис. 1; теорема 1.

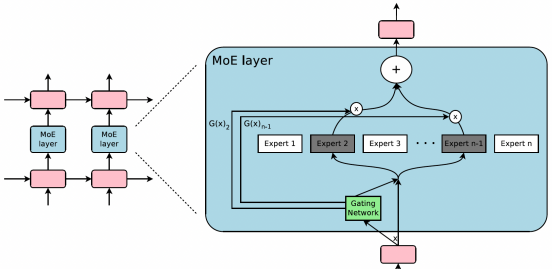
Разреженное внимание



- В причинной маске токен видит всё прошлое. Разреженная маска оставляет часть: полосу соседей и позиции с шагом. Стоимость $O(N\sqrt{N})$ [85]; скользящее окно w с глобальными токенами — $O(N \cdot w)$ [87].
- Разреженное внимание, обучаемое вместе с моделью (2025): сжатие блоков, выбор важных блоков, скользящее окно; на 64K токенов прямой проход быстрее в 9 раз [109].
- Дальние токены связаны через несколько слоёв: часть пар напрямую не видит друг друга.

Источник рисунка: [85] R. Child и др., «Generating long sequences with sparse transformers», 2019, рис. 3. [87] I. Beltagy и др., «Longformer», 2020. [109] J. Yuan и др., «Native sparse attention», ACL 2025.

Смесь экспертов: много параметров, мало вычислений



$$y = \sum_i G(x)_i \, E_i(x), \qquad G(x)_i = 0 \;\Rightarrow\; E_i \text{ не вычисляется.}$$

- Qwen3 235B: 128 экспертов, 8 активны, ≈22 млрд активных [119].
- Считается мало, хранится всё: память ускорителя — по полному числу параметров [104].
- Разрежены веса, внимание не меняется [89]: удешевляет токен и не расширяет окно.

Источник рисунка: [83] N. Shazeer и др., «Outrageously large neural networks: The sparsely-gated mixture-of-experts layer», ICLR 2017, рис. 1; G и E_i — обозначения статьи. [104] A. Q. Jiang и др., «Mixtral of experts», 2024 (не рецензируемое). [89] W. Fedus и др., «Switch Transformers», JMLR 2022, §7.

Сводка: что снимает каждый приём

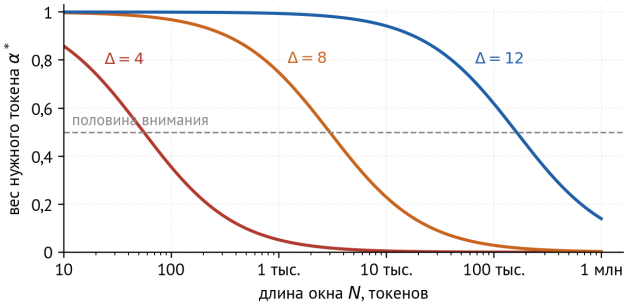
Приём	**Что уменьшает**	**Что остаётся**
FlashAttention	память $O(N^2) \to O(N)$	операций $O(N^2 d)$
разреженное внимание	число операций	не все пары токенов связаны
MQA, GQA, MLA	KV-кэш	рост кэша по N
вытеснение из кэша	KV-кэш	вытесненное недоступно
смесь экспертов	вычисления на токен	внимание и окно без изменений
расширение окна (YaRN)	заявленную длину	эффективная длина

[101] G. Xiao и др., «Efficient streaming language models with attention sinks», ICLR 2024. [98] B. Peng и др., «YaRN: Efficient context window extension», ICLR 2024. Остальные — на предыдущих слайдах.

Разбавление внимания

Выкладка в упрощённой модели: в окне один нужный токен с оценкой z^* и $N-1$ остальных с одинаковой оценкой z; отрыв $\Delta = z^* - z$. Вес нужного токена после softmax:

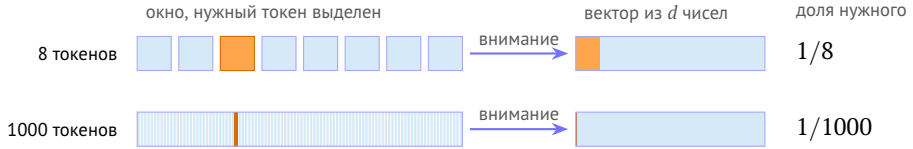
$$\alpha^* = \frac{e^{z^*}}{e^{z^*} + (N-1)\,e^{z}} = \frac{1}{1 + (N-1)\,e^{-\Delta}}, \qquad \alpha^* \geq \tfrac{1}{2} \iff \Delta \geq \ln(N-1).$$



Чтобы удержать внимание на нужном токене, отрыв оценки должен расти как логарифм длины окна. Оценки ограничены весами и точностью вычислений, поэтому с ростом N вес нужного токена падает.

Коллапс представлений

Следующий токен предсказывается по одному вектору — представлению последнего токена. В этот вектор из d чисел сводится всё окно; размер вектора от длины окна не зависит.



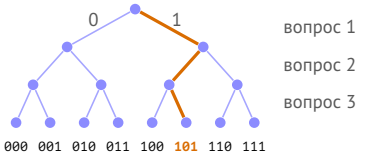
- Упрощённо: веса внимания равны, вектор — среднее токенов окна, доля каждого токена — $1/N$.
- Окна с нужным токеном и без него дают векторы, которые различаются на величину порядка $1/N$. Относительная точность чисел bfloat16 — около $2^{-8} \approx 0{,}4\,\%$: при $N = 1000$ различие в $0{,}1\,\%$ теряется при округлении, и модель отвечает на оба окна одинаково.

Чем больше окно, тем меньше в итоговом векторе доля каждого токена и тем труднее модели найти в окне нужное.

[107] F. Barbero и др., «Transformers need glasses! Information over-squashing in language tasks», NeurIPS 2024 — строгая формулировка для трансформера: теор. 4.2.

Сколько бит стоит токен

Сжать текст — записать его строкой из 0 и 1, по которой текст восстанавливается без потерь. Один бит — ответ на один вопрос «да или нет».



8 равновероятных вариантов: 3 вопроса, 3 бита.

$$I(x) = -\log_2 p(x) \text{ бит}$$

$p(x)$ — вероятность исхода x

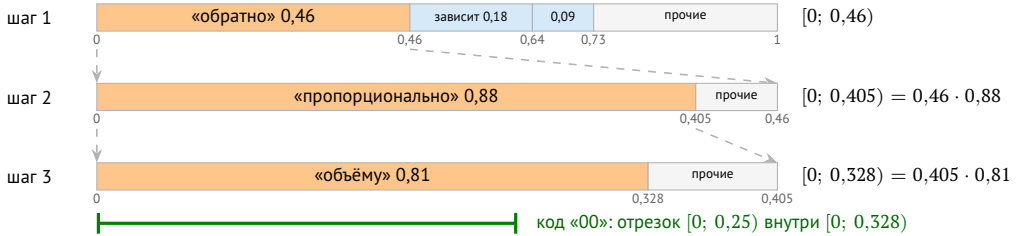
Исход	p	бит
монета: орёл	$1/2$	1
кубик: шестёрка	$1/6$	2,58
токен словаря Qwen3 наугад	$1/151\,669$	17,2
после «Давление газа при постоянной температуре»		
«обратно»	0,46	1,12
«пропорционально»	0,88	0,18
«объёму»	0,81	0,30
«плотности» вместо «объёму»	0,04	4,64

Без модели каждый токен словаря стоит 17,2 бита. Модель делает ожидаемый токен дешёвым, неожиданный — дорогим. Среднее $-\log_2 p$ по тексту — кросс-энтропия, функция потерь при обучении модели [100].

[73] C. E. Shannon, «A mathematical theory of communication», Bell Syst. Tech. J., 1948: средняя длина кода не меньше среднего $-\log_2 p$. [100] G. Delétang и др., «Language modeling is compression», ICLR 2024 — §2. Вероятности условные.

Арифметическое кодирование

Продолжение «обратно пропорционально объёму» записывается двумя битами. Кодер и декодер пользуются одной моделью; чем точнее её предсказание, тем короче код.

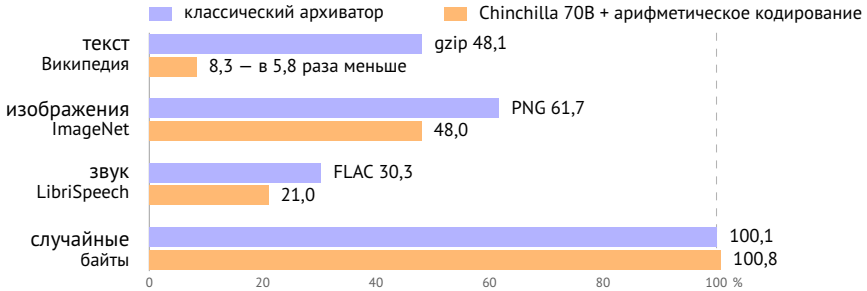


- На каждом шаге текущий отрезок делится между возможными токенами пропорционально их вероятностям; остаётся кусок записанного токена. Ширина итогового отрезка ρ — произведение вероятностей.
- Код — самая короткая двоичная дробь, отрезок которой лежит внутри итогового: «00». Декодер с той же моделью восстанавливает по нему три токена. Длина кода не больше $\lceil -\log_2 \rho \rceil + 1$ бит [100, §2].

[100] G. Delétang и др., «Language modeling is compression», ICLR 2024 — §2, рис. 1; ρ — обозначение статьи. Вероятности условные.

Языковая модель как архиватор

Опыт: по 1 ГБ данных каждого вида, сжатие кусками по 2048 байт. Длина столбца — размер после сжатия в процентах от исходного; меньше — лучше.



- Chinchilla 70B обучена на тексте, но изображения и звук сжимает лучше PNG и FLAC: закономерности нового вида данных модель подхватывает в пределах окна.
- Случайные байты не сжимает ни один способ: предсказывать в них нечего. Размер самой модели в эти числа не входит.

[100] G. Delétang и др., «Language modeling is compression», ICLR 2024 — табл. 1: сырая степень сжатия, куски по 2048 байт.

Что из этого следует

1. Контекст сжимает

gzip, сжимая текст кусками по 2048 байт, оставляет 48,1 %; целым файлом — 32,3 %. Модель тоже сжимает тем лучше, чем больше текста видит в окне.

2. Знание — в весах

Если добавить к сжатому тексту размер самой модели, Chinchilla 70B превращает 1 ГБ текста в 140 ГБ: веса больше данных. Общее знание модель приносит с собой.

3. Ожидаемое бесплатно

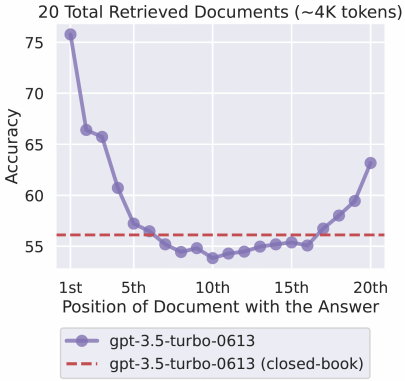
Строка, которую модель и так предсказывает, стоит доли бита: она почти не добавляет информации. Много бит несёт то, чего модель предсказать не может.

Интерпретация: в окно стоит класть то, чего модель не знает, — особенности своего проекта; общее знание модель уже несёт в весах.

---

[100] G. Delétang и др., «Language modeling is compression», ICLR 2024 — табл. 1 (сжатие кусками и целиком; с учётом размера модели — 14 008 %), рис. 4.

Потерянная середина



Вопрос и 20 документов (≈4K токенов), ответ — в одном из них. Меняется только позиция нужного документа.

Позиция	Точность
1-я	75,8 %
10-я	53,8 %
20-я	63,2 %
без документов	56,1 %

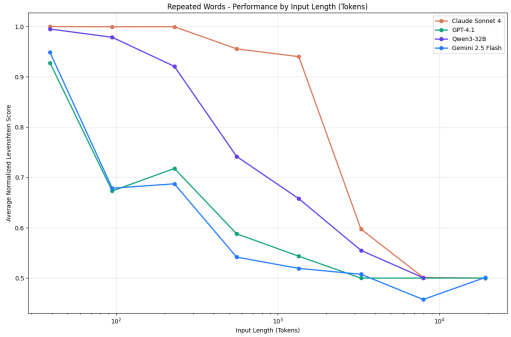
Документ в середине окна полезен меньше, чем отсутствие документов. Версии моделей с расширенным окном ведут себя почти так же.

Источник рисунка: [97] N. F. Liu и др., «Lost in the middle: How language models use long contexts», TACL 2024 — рис. 1, табл. 1, табл. 6.

Заявленное и эффективное окно

Модель	Заявлено	Эффективно
GPT-4	128K	64K
Llama3.1-70B	128K	64K
Qwen2-72B	128K	32K
Yi-34B	200K	32K
GLM4-9B	1M	64K
LWM	1M	<4K

Эффективная длина — наибольшая, на которой среднее по 13 задачам выше 85,6 %. В простом поиске «иголки» почти все модели дают около 100 % на всей заявленной длине [105].



Повтор последовательности слов: качество падает уже к тысячам токенов [110].

[105] C.-P. Hsieh и др., «RULER: What's the real context size of your long-context language models?», COLM 2024 — табл. 3, табл. 10. Источник рисунка: [110] K. Hong, A. Troynikov, J. Huber, «Context rot», Chroma Research, 2025 (не рецензируемое).

Вопрос до и после данных

извлечение по ключу: вопрос стоит дважды

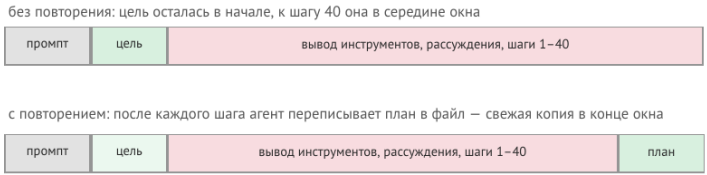
```
Найди значение по ключу "9f2c...".
{"a41e...": "7d10...", "15d0...": "e77a...", ... 75 пар ...,
 "9f2c...": "c3b8...", "0b7a...": "51fe...", ...}
Ключ: "9f2c...". Значение:
```

- Из-за причинной маски токены данных обрабатываются, пока вопроса «ещё нет». Вопрос в начале позволяет модели читать данные вместе с ним.
- Результат зависит от задачи: извлечение по ключу становится почти безошибочным, вопросы по документам — почти без изменений.
- Для запроса агенту: задача и ограничения — в начале, повтор критерия готовности — в конце.

[97] N. F. Liu и др., «Lost in the middle», TACL 2024 — §4.2, рис. 9. Ключи сокращены; в опыте — 75, 140 и 300 пар.

Как держать цель на виду в длинной сессии

Потерянная середина касается и самой задачи агента: цель записана в начале окна, а через несколько десятков шагов её окружает вывод инструментов.



- Приём: после каждого шага агент переписывает план в файл; свежая копия плана оказывается в конце окна [111].
- Правило в AGENTS.md: «после каждого блока обнови `STATUS.md`: что сделано, что осталось, критерий готовности».

STATUS.md после шага 40

```
Сделано: серии 1-3, проверка пройдена
Осталось: серия 4, отчёт
Готово, когда: checks/check.py -> 0
```

[111] Y. Ji, «Context engineering for AI agents: Lessons from building Manus», 2025 (не рецензируемое): в среднем около 50 вызовов инструментов на задачу.

Окно как состояние

Состояние агентной системы — пара (s_t, e_t): окно и среда. Модель видит только окно s_t.

- Всё, что агент «знает» о проекте в данный момент, — веса модели плюс окно s_t. Файл на диске существует для агента, только когда его содержимое прочитано в окно.
- Чтение файла переносит часть среды e_t в окно s_t; запись переносит часть окна в среду.
- Сжатие окна заменяет s_t сводкой: разные истории могут дать одну сводку, и следующее действие может измениться.

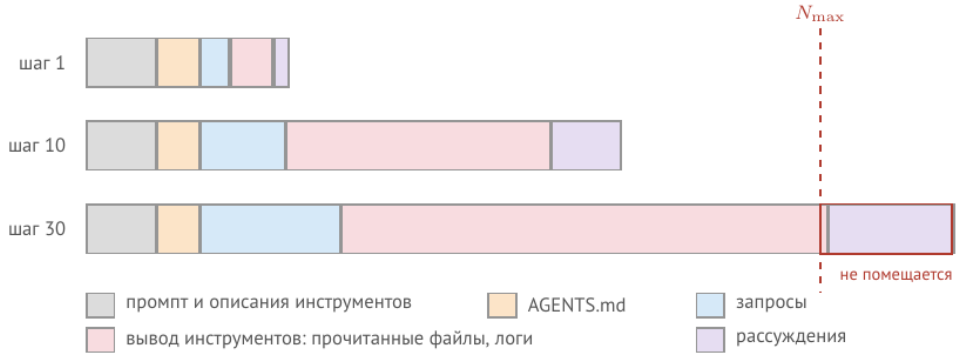
Управлять агентом — значит управлять тем, что попадает в окно и что из него уходит.

Из чего склеено окно



Для модели все части — одна строка токенов. Ведущее приложение склеивает их в этом порядке на каждом вызове; постоянное начало кэшируется, хвост растёт.

Прочитал — значит положил в окно



- Каждый вызов инструмента дописывает вывод в окно где он остается до конца сессии или до сжатия.
- Измерение на 857 сессиях агента: 79,4 % байтов разговора — вывод инструментов.

[118] T. Mason, «The missing memory hierarchy: Demand paging for LLM context windows», 2026, §5.1 (сессии одного пользователя; не рецензируемое). Доли на схеме условные.

Какие инструменты агент вызывает на самом деле

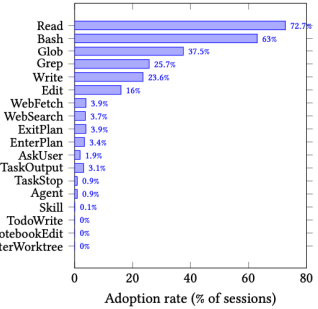
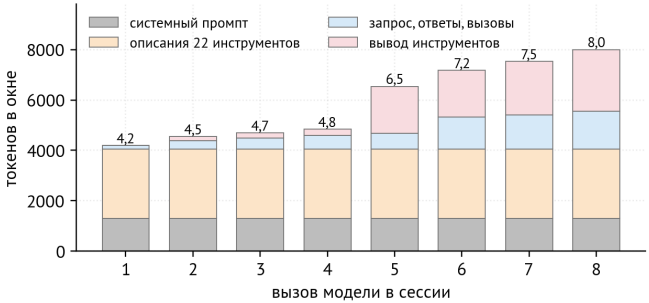


Figure 1. Tool adoption rate across 801 active sessions. The median session uses 3 of 18 available tools. Seven tools see zero or near-zero adoption, yet their full schemas (collectively ~24,500 bytes) are sent on every API call.

- 801 сессия агента программирования: медиана — 3 инструмента из 18.
- Семь инструментов почти не вызываются, но их полные описания (около 24,5 тыс. байт) уходят в каждый запрос к модели.
- Описание инструмента — часть окна на каждом шаге, вызывают его или нет. У учебного агента 22 инструмента; как это выглядит в токенах — на следующем слайде.

Источник рисунка: [118] T. Mason, «The missing memory hierarchy: Demand paging for LLM context windows», 2026, рис. 1 (сессии одного пользователя; не рецензируемое).

Окно учебного агента: измерение



- Тестовый проект `spectra_peaks` без AGENTS.md, модель Qwen3 235B: до начала работы в окне 4,2 тыс. токенов — системный промпт (1,3 тыс.) и описания 22 инструментов (2,7 тыс.), из них 7 — инструменты Lean, в этой задаче не нужные.
- За 8 вызовов модели ввод составил 47,5 тыс. токенов при ответе в несколько строк. Запуск `ai4science -no-lean` убирает описания инструментов Lean из каждого вызова.

Измерение курса: журналы запросов учебного агента, токенизатор Qwen3, 24.09.2026. [121] Репозиторий учебного агента, README.

Слепое пятно

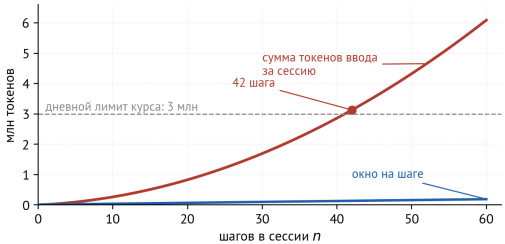


Решение может быть верным в пределах окна и ошибочным для проекта. После сжатия или обрыва сессии из окна выпадают решения, принятые в разговоре; сохраняется то, что записано в файлах, которые агент перечитывает.

Стоимость сессии растёт квадратично

На каждом шаге модели заново подаётся всё окно. Блок m_k, дописанный на шаге k, входит во все следующие окна — его оплачивают $n - k + 1$ раз:

$$\sum_{t=1}^{n} |s_t| \;=\; n\,|s_0| + \sum_{k=1}^{n} (n-k+1)\,|m_k| \;\le\; n\,|s_0| + \delta\,\frac{n(n+1)}{2} \;=\; O(n^2), \qquad |m_k| \le \delta.$$



Ранние блоки — самые дорогие: файл, прочитанный на первом шаге, оплачивается на каждом следующем. Условный пример на графике: 10 тыс. токенов на старте, 3 тыс. за шаг — дневной лимит курса исчерпывается за 42 шага одной сессии.

Кэш префикса

KV-кэш годен только для неизменного начала последовательности. Провайдер хранит его между вызовами: повторное чтение начала окна стоит около десятой части обычного ввода.

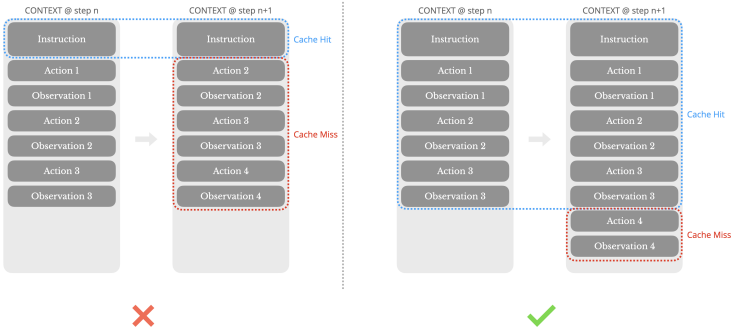
- В одной сессии агента чтение из кэша — половина стоимости уже к 27 500 токенам и 87 % к концу [116].
- Отношение ввода к выводу в агенте: около 100:1 [111]; 933:1 и повторная обработка каждого байта вывода инструментов по медиане 84 раза [118].
- Изменение одного токена в начале окна делает кэш недействительным [111].

Начало окна держится неизменным: системный промпт, AGENTS.md, описания инструментов. Новое дописывается в конец.

---

[116] P. Zeyliger, «Expensively quadratic: The LLM agent cost curve», exe.dev, 2026. [111] Y. Ji, «Context engineering for AI agents: Lessons from building Manus», 2025. [118] T. Mason, 2026, §5.2. Все три — не рецензируемые.

Как не сбить кэш префикса

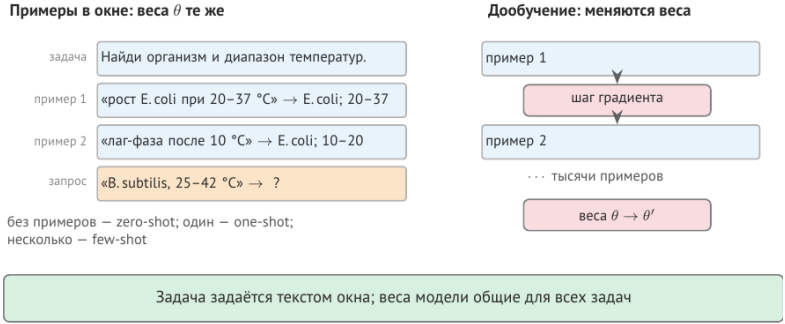


- в системном промпте нет даты и времени;
- окно только дописывается, прошлые шаги не правятся;

- JSON сериализуется с одним порядком ключей;
- инструменты не удаляются посреди работы: вызов запрещает ведущее приложение.

Источник рисунка: [111] Y. Ji, «Context engineering for AI agents: Lessons from building Manus», 2025 (не рецензируемое). Слева — правка начала: промах кэша; справа — дописывание: попадание.

Дообучение или примеры в окне



Дообучение: меняются веса; нужны сотни и тысячи примеров и вычисления; получается новая модель. **Примеры в окне:** веса не меняются; действуют, пока примеры в окне, и стоят токенов на каждом вызове.

Схема по [88] T. B. Brown и др., «Language models are few-shot learners», NeurIPS 2020, рис. 2.1; примеры — из тестового проекта `lit_review`.

Без примеров, с одним, с несколькими

zero-shot

```
Переведи в СИ.
2,5 атм =>
```

one-shot

```
Переведи в СИ.
1 атм => 101325 Па
2,5 атм =>
```

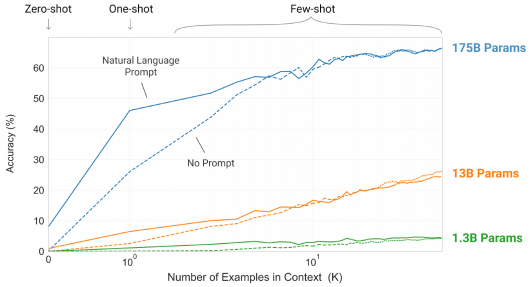
few-shot

```
Переведи в СИ.
1 атм => 101325 Па
760 мм рт. ст. => 101325 Па
1 бар => 100000 Па
2,5 атм =>
```

- Без примеров модель получает только описание задачи: форма ответа не задана.
- Пример задаёт формат: единицы, запись чисел, одна строка без пояснений.
- Несколько разных примеров показывают, что меняется от строки к строке, а что остаётся.

Постановка по [88] T. B. Brown и др., «Language models are few-shot learners», NeurIPS 2020, §2, рис. 2.1.

Сколько дают примеры



Задача — восстановить слово, в которое вставлены случайные символы. Точность растёт с числом примеров в окне, и тем сильнее, чем крупнее модель.

Окно GPT-3 — 2048 токенов, поэтому примеров в запросе 10–100: число примеров ограничено окном.

Источник рисунка: [88] T. B. Brown и др., «Language models are few-shot learners», NeurIPS 2020 — рис. 1.2, §2.

Почему агент универсален

- Одна модель решает разные задачи: задача задаётся текстом окна, веса общие.
- Агент для программирования — частный случай: спецификация задачи, файлы проекта и вывод инструментов в окне превращают модель общего назначения в исполнителя конкретной работы.
- Сменили AGENTS.md и запрос — получили другого агента на той же модели.

Исследователь управляет агентом через окно: спецификацией, примерами и тем, что агент прочитает.

Как давать примеры агенту

однообразные примеры

```
a.csv -> 12 строк, 3 столбца
b.csv -> 12 строк, 3 столбца
c.csv -> 12 строк, 3 столбца
d.csv ->
```

разные типичные примеры

```
a.csv -> 12 строк; T, p; пропусков нет
spectrum.txt -> 601 точка; 2 пика
notes.md -> текст; 2 раздела
d.csv ->
```

- Слева продолжить шаблон проще, чем открыть файл: однообразные примеры и повторяющийся формат ведут агента к копированию [111].
- В тесте длинного контекста больше 80 % ответа модели в задаче подсчёта частых слов на 128K — копия примера из запроса [105].
- Справа примеры различаются тем, что нужно смотреть в файле, и совпадают формой ответа [113].

---

[111] Y. Ji, Manus, 2025. [113] P. Rajasekaran и др., Anthropic, 2025. Оба не рецензируемые. [105] C.-P. Hsieh и др., «RULER», COLM 2024, §5.

Когда окно переполняется

Спецификация, примеры и вывод инструментов наполняют окно; длинная работа его переполняет. Три пути:

Сжать

каскадное сжатие со сверкой; уплотнение окна агента

Вынести

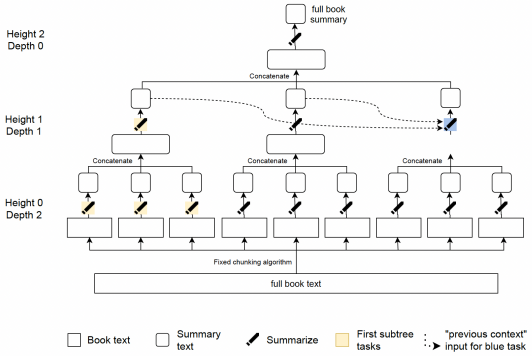
записки и файлы состояния; иерархия памяти; указатель и пометки

Разделить

подзадачи в отдельных окнах; в основное окно — сводка

Каждый путь теряет часть окна; поэтому каждый требует проверки того, что осталось.

Иерархическое сжатие

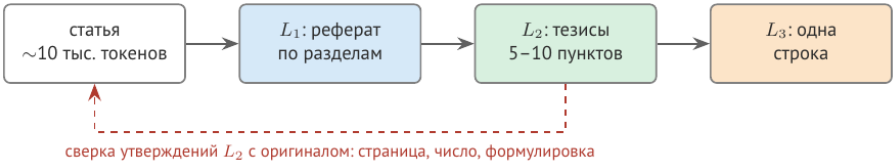


Книга режется на куски по ≈600 токенов; сводки склеиваются и сжимаются снова, в 5–10 раз на шаге.

Ошибка из статьи: в «Гордости и предубеждении» приглашение на танец превратилось в предложение руки.

Источник рисунка: [90] J. Wu и др., «Recursively summarizing books with human feedback», 2021 — рис. 1, прил. A, §6.1. В курсе метод называется каскадным сжатием.

Каскадное сжатие статьи: пример



L_3 — одна строка

Модели хуже используют сведения из середины длинного контекста:
точность U-образно зависит от позиции нужного документа.

L_2 — тезисы (фрагмент)

1. Точность наибольшая, когда ответ в начале или в конце контекста.
2. В середине точность ниже, чем без документов вовсе: 53,8 против 56,1 %.
3. Модели с расширенным окном ведут себя почти так же, как исходные.
4. Вопрос до и после данных устраняет эффект во всех задачах.

Уплотнение окна агента

- **Очистка вывода инструментов**: прочитанный файл заменяется ссылкой «перечитать при необходимости».
- **Сводка у предела окна**: история сжимается, работа продолжается.
- **Заметки вне окна**: план и ход работы в файле; обновлённый список задач в конце окна удерживает цель у конца, где её лучше видно.
- **Ошибки остаются в окне**: запись о неудачной попытке снижает её повторение.

---

[113] P. Rajasekaran и др., «Effective context engineering for AI agents», Anthropic, 2025. [111] Y. Ji, «Context engineering for AI agents», Manus, 2025. Оба не рецензируемые.

Окно в учебном агенте

Команда или флаг	Что делает с окном
`/status`	показывает модель, расход токенов и заполнение окна
`/compact`	сжимает историю сессии; автоматически — примерно при 100 тыс. токенов
`/clear`, `/new`	очищают историю; новая задача — новое окно
`ai4science run "..."`	одна задача в отдельном процессе с чистым окном
`-no-lean`	убирает описания инструментов Lean
`AI4SCIENCE_WEB_DISABLED=1`	убирает инструменты поиска в сети

После смены модели командой `/model` история сохраняется, окно не пересчитывается: история должна помещаться в самое короткое окно списка моделей (128K).

[121] А. П. Халов, О. М. Атаева, «AI4Science: консольный агент курса», GitHub, 2026 — README, разделы «Команды в сессии», «Смена модели».

Записки по дороге: память, которую ведёт исследователь

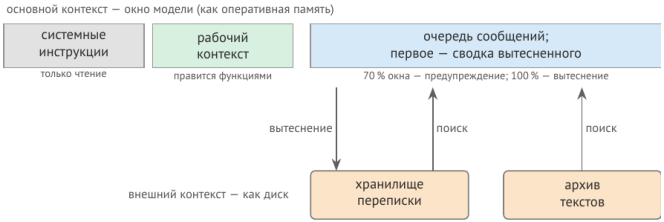
- `diary.md` — только дописывается: решения, проверки, итоги.
- `STATUS.md` — перезаписывается: где работа сейчас, что дальше, какие ограничения действуют.
- Сообщения коммитов — почему сделано изменение.
- В AGENTS.md — правило: прочитать `STATUS.md` в начале, обновить после каждого блока.

STATUS.md

```
## Где мы
Аппроксимация серий 1-3 готова, проверка пройд
    ена.
## Дальше
Серия 4: сначала сверить единицы (Цельсий или
    кельвин).
## Действует
- data/raw только чтение
- серию 2024-03 не использовать
```

Автоматическое сжатие решает за исследователя; записки фиксируют то, что исследователь считает важным.

MemGPT: окно как оперативная память



- Окно разделено на три части: системные инструкции, рабочий контекст, который модель правит сама, и очередь сообщений.
- При 70 % заполнения модель получает предупреждение; при 100 % часть сообщений вытесняется во внешнее хранилище, в очереди остаётся их сводка.
- Вытесненное доступно через поиск: модель сама вызывает функции подкачки.

Схема по [102] C. Packer и др., «MemGPT: Towards LLMs as operating systems», 2023, §2, рис. 3. Сравнение с памятью ОС — аналогия.

Подкачка: что показали измерения

MemGPT (2023)

- ответы по памяти прошлых сессий, GPT-4: со сводкой прошлых разговоров — 32,1 %, с подкачкой — 92,5 %;
- сводка теряет детали, поиск их возвращает.

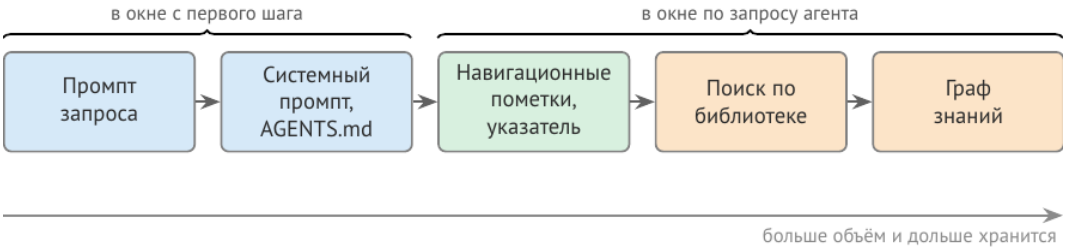
Агент программирования (2026)

- устаревший вывод инструментов заменяется меткой «перечитать при необходимости»;
- ввод меньше на 37 %, задача решена;
- повторные чтения — 0,025 % вытеснений.

---

[102] C. Packer и др., «MemGPT», 2023, табл. 2. [118] T. Mason, «The missing memory hierarchy», 2026, табл. 4–5 (сессии одного пользователя; не рецензируемое).

Лестница памяти



- Чем выше ступень, тем больше объём и тем реже содержимое попадает в окно целиком.
- Подгрузка по ссылке: агент держит в окне пути к файлам и запросы, а содержимое читает, когда оно нужно.

[113] P. Rajasekaran и др., «Effective context engineering for AI agents», Anthropic, 2025 (не рецензируемое).

Файл-указатель

INDEX.md — файл-указатель проекта

```
# Где что лежит
- data/raw/      измерения, только чтение; описание в data/README.md
- src/io.py      чтение и запись файлов: load_rates(), save_table()
- src/physics.py формулы: arrhenius(), fit_arrhenius()
- results/       результаты; каждый файл пишет один скрипт из src/
- memory/L1.md   сжатие статьи; дальше L2.md, L3.md
- STATUS.md      где работа сейчас; diary.md -- журнал
```

- Агент читает указатель и открывает только нужный файл: в окно попадает путь к файлу, остальной проект остаётся на диске.
- AGENTS.md ссылается на указатель: «где что лежит — в `INDEX.md`».

Пометки в коде

src/physics.py — строка документации ведёт дальше

```
def fit_arrhenius(T, k):
    """Аппроксимация k(T) законом Аррениуса; T в кельвинах.

    Данные: массивы из src/io.py::load_rates().
    Проверка: tests/test_physics.py::test_known_activation_energy.
    Используется в: src/report.py::make_table().
    """
```

- Каждая строка документации указывает на следующие файлы: агент идёт по ссылкам, не читая проект целиком.
- Агента можно попросить писать такие пометки самому: следующая сессия пойдёт по ним.
- Пометки образуют служебный граф проекта: связи придуманы по ходу работы, типы связей не заданы, противоречия не проверяются.

Действия, вычисления, данные

всё в одной функции

```
def process():
    df = pd.read_csv("data/raw/k.csv")
    df["k"] = A * np.exp(-Ea / (R * df.T))
    df.to_csv("results/k.csv")
    print("готово")
```

вычисление отдельно

```
def arrhenius(T, A, Ea):
    return A * np.exp(-Ea / (R * T))

def run():
    df = load_rates()
    save_table(arrhenius(df.T, A, Ea))
```

- **Действие** зависит от момента и среды: чтение и запись файлов, печать, сеть.
- **Вычисление** зависит только от аргументов: `arrhenius` даёт один результат при одних входах.
- **Данные** — то, что лежит в файлах и передаётся между функциями.

---

[120] E. Normand, «Grokking Simplicity: Taming Complex Software with Functional Thinking», Manning, 2021.

Проект как библиотека функций

структура

```
src/io.py        действия: файлы, сеть
src/physics.py   вычисления: чистые функции
src/report.py    действия: сборка таблиц
data/raw/        данные: только чтение
tests/           проверки вычислений
```

AGENTS.md, раздел 5

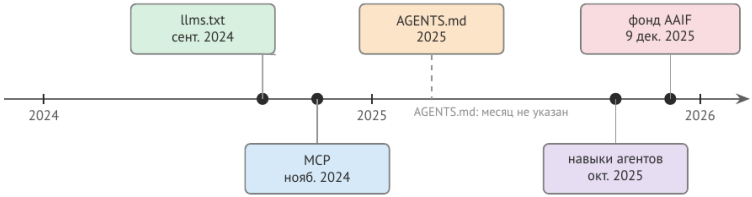
```
- Новая формула -- чистая функция
  в src/physics.py с тестом.
- Файлы читать и писать только
  через src/io.py.
```

- Вычисление проверяется тестом без среды: входы и ожидаемый результат.
- Для окна: агент читает сигнатуру и строку документации вместо всего скрипта и переиспользует готовую функцию вместо того, чтобы писать новую.

---

[120] E. Normand, «Grokking Simplicity», Manning, 2021.

Открытые спецификации для агентов



- `llms.txt` — указатель сайта для модели; MCP — протокол подключения инструментов; AGENTS.md — файл инструкций проекта; навыки — папки с инструкциями, которые агент подгружает по мере надобности.
- Все четыре решают одну задачу: что положить в окно агента заранее и что оставить на подгрузку.

[108] J. Howard, «The /llms.txt file», 2024. [70] Anthropic, «Introducing the Model Context Protocol», 2024. [71] «AGENTS.md», agents.md. [114] Anthropic, «Agent Skills», 2025.

Agentic AI Foundation



9 декабря 2025 г. Linux Foundation создал фонд Agentic AI Foundation. Первые проекты фонда:

- MCP — передан Anthropic;
- goose — передан Block; на нём работает учебный агент курса;
- AGENTS.md — передан OpenAI.

По сообщению фонда, опубликовано более 10 000 серверов MCP, AGENTS.md используют более 60 000 открытых проектов.

---

[115] The Linux Foundation, «Linux Foundation announces the formation of the Agentic AI Foundation», пресс-релиз, 09.12.2025. Изображение — из пресс-релиза.

AGENTS.md: что он меняет и чего не меняет

Файл в папке проекта, который агент читает в начале каждой сессии. Он входит в s_0 и тем самым задаёт политику: цель, критерий готовности, протокол, память.

Меняет

образ результата; порядок работы; что читать первым; какие ошибки не повторять

Не меняет

множество доступных действий $\mathcal{A}$: запрет в файле действует на слово

- Формат открытый; файл известен под именами `AGENT.md`, `AGENTS.md`, `CLAUDE.md`, `.cursorrules`.
- Для одноразовой сессии файл не нужен, но цель и критерий готовности нужны и там — в самом запросе.

---

[71] «AGENTS.md — open format for guiding coding agents», Agentic AI Foundation, `https://agents.md/` — открытый формат файла-инструкции для агентов.

Структура AGENTS.md



- Разделы 0, 1, 4, 5, 7 описывают задачу: образ результата, проверку, запреты, режим работы, остановку.
- Разделы 2, 3, 6 — ориентацию в проекте: где что лежит, чем пользоваться, что помнить между сессиями; сюда входит «чего агент не может знать сам».

AGENTS.md стартер пак

Пять образцов по шаблону курса, в сокращённом виде. Каждый описывает цикл одного вида работы: образ результата, критерий готовности, режим, запреты по трём пространствам, остановку.

Работа	Пример	Режим	Критерий готовности
обзор литературы	биология	конвейер	у каждой ссылки есть PDF в библиотеке, DOI совпадает
рецензирование	химия	план и исполнение	все разделы шаблона; у слабых сторон — страница рукописи; ≥ 5 проверенных утверждений
обработка данных	науки о Земле	ReAct	тесты на синтетическом ряде + сверка трёх станций с ручным расчётом
подбор параметров	физика	поиск с метрикой	ошибка на контрольной задаче, которой агент не видел
текст статьи	математика	план и исполнение	сборка, объём, каждое число есть в `results/`

- Пометки запретов: `[окно]` — на слово; `[вызов]` — ведущее приложение проверяет вызов до исполнения; `[среда]` — права доступа, отдельная копия, инвариант.
- Исходные данные и необратимые действия закрываются на уровне `[вызов]` или `[среда]`.

Как писать AGENTS.md

- **Проверка строки:** «если её убрать, ошибётся ли агент?» Если нет — строка вычёркивается: в длинном файле правила теряются.
- **Включать:** команды запуска, тесты и способ их запуска, особенности окружения, известные ловушки — то, что нельзя угадать по коду.
- **Не включать:** то, что видно из кода, и общие пожелания вроде «работай аккуратно».
- **Однозначность:** инструкции декодируются заново каждую сессию, противоречия между разделами накапливаются в ошибки.

Дайте агенту способ проверить работу самому — тест, сборку, сверку с эталоном. Без этого «выглядит готовым» остаётся единственным сигналом.

---

[72] Anthropic, «Claude Code best practices», официальная документация (не рецензируемое) — рекомендации к файлу AGENTS.md.

Почему коротко и ясно

Рекомендации к AGENTS.md выводятся из нижних уровней:

1. каждая строка входит в начало окна и оплачивается при каждом вызове модели;
2. каждая строка делит внимание с данными задачи — разбавление внимания;
3. длинный файл отодвигает данные к середине окна;
4. строка, которую модель и так предсказывает, почти не добавляет информации; строка о том, чего модель знать не может, добавляет;
5. пересказ README и кода не повышает долю решённых задач и удорожает работу [117].

---

[117] T. Gloaguen и др., «Evaluating AGENTS.md», 2026, §5. Пункт 4 — интерпретация через сжатие [100].

Правило строки: AGENTS.md до и после

Правило строки: для каждой строки AGENTS.md спросить — если её убрать, ошибётся ли агент? Если нет, строка лишняя [72].

до: пересказ и пожелания

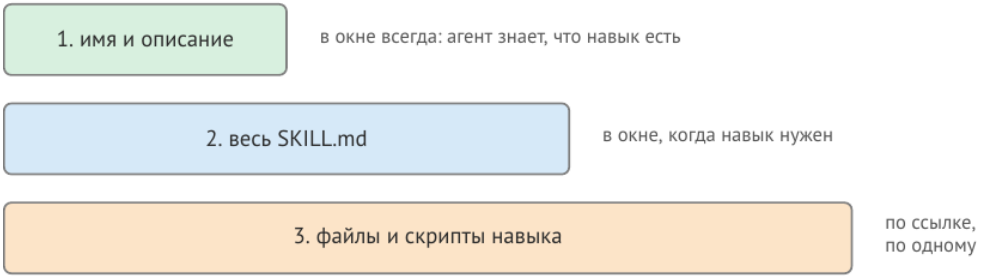
```
## О проекте
Проект о кинетике реакций.
Python 3.11, numpy, pandas.
Код в src/, данные в data/.
## Правила
- Пиши аккуратный код.
- Проверяй результаты.
```

после: то, чего агент не узнает сам

```
## 1. Критерий готовности
pytest tests/ -q -- все зелёные.
## 2. Чего агент не знает сам
- В data/raw температуры по Цельсию,
  в формулах -- в кельвинах.
- Серию 2024-03 не использовать:
  неисправный термостат.
## 4. Запреты
- Не изменять data/raw/. [среда]
```

- Слева: тему проекта и библиотеки агент увидит в коде; «аккуратно» и «проверяй» не проверяемы — убрав эти строки, агент не ошибётся.
- Справа: без любой из строк агент ошибётся — спутает единицы, возьмёт плохую серию, сочтёт работу готовой без тестов.

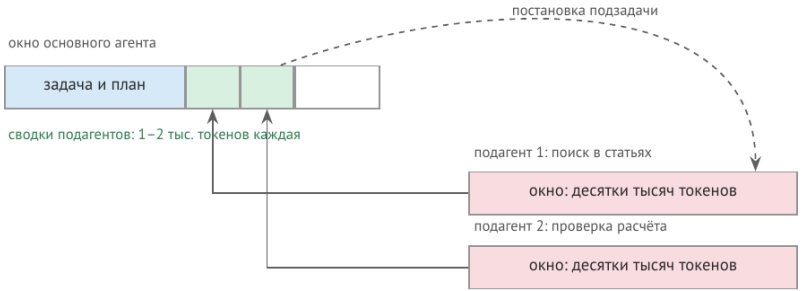
Навыки: постепенное раскрытие



- Навык — папка с инструкцией `SKILL.md` и файлами; в окне всегда только имя и описание, остальное читается по мере надобности [114].
- Приём переносится на свой проект: в AGENTS.md — короткие правила и ссылки, подробности — в файлах, на которые он указывает.

[114] Anthropic, «Equipping agents for the real world with Agent Skills», 2025; открытый стандарт с 18.12.2025 (не рецензируемое).

Подагент: отдельное окно на подзадачу



- Подагент получает подзадачу и чистое окно, тратит десятки тысяч токенов и возвращает сводку в 1–2 тыс. токенов [113].
- Вывод инструментов у подагентов обрабатывается повторно по медиане 12,8 раза, в основной сессии — 84,4 [118]. Общий расход растёт, окно основного агента остаётся коротким.

[113] P. Rajasekaran и др., «Effective context engineering for AI agents», Anthropic, 2025. [118] T. Mason, 2026, §5.2. Оба не рецензируемые.

Подзадачи в учебном агенте

каждый run — отдельное окно

```
for f in paper/sections/*.md; do
  ai4science run "Сожми $f до 150 слов; числа и формулы -- дословно; \
    запиши в memory/L1/$(basename $f)"
done
ai4science run "Собери memory/L1/*.md в 5-10 тезисов: memory/L2.md"
```

- Каждый раздел статьи сжимается в своём окне; итоговый шаг читает только сводки.
- Сводки лежат в файлах: их можно сверить с оригиналом и перезапустить отдельный шаг.
- Общий расход токенов — сумма запусков; окно каждого запуска короткое.

Правила управления окном

1. Задача и ограничения — в начале запроса, критерий готовности — в конце.
2. В AGENTS.md — только то, чего агент не узнает сам; подробности — в файлах по ссылкам.
3. Лишние инструменты выключены: `-no-lean`, без поиска в сети, если он не нужен.
4. Большие файлы не читаются целиком: указатель, поиск, выдержки.
5. Длинная работа — с записками вне окна: `STATUS.md`, `diary.md`; сжатие — со сверкой.
6. Независимые подзадачи — в отдельных запусках, результат — сводкой в файл.
7. Начало окна не меняется посреди сессии.

Самостоятельная практика недели 3

Нагрузка 3–5 часов на своём материале. Стартовый набор — в общей папке курса.

1. **Ноутбук «Агент в коде»** (45 мин). Прогон по разделам, затем своя задача со своим критерием готовности.
2. **Каскадное сжатие своей статьи** (90 мин). L_1 — реферат по разделам, L_2 — 5–10 тезисов, L_3 — одна строка; не меньше трёх утверждений L_2 сверены с оригиналом, со страницей.
3. **AGENTS.md целиком** (45 мин). Разделы 2, 3, 6: карта проекта, инструменты, память; раздел «чего агент не может знать сам».
4. **Файл-указатель и пометки** (30 мин). `INDEX.md` и пометки в коде, которые ведут к нужным файлам. Коммит.

Артефакт недели 3

Что сделать

Трёхуровневое сжатие своей статьи со сверкой утверждений; рабочий AGENTS.md и файл-указатель для своего проекта.

Минимум для «зачтено»

- уровни $L_1 \to L_2 \to L_3$;
- не меньше трёх утверждений сверены с оригиналом;
- AGENTS.md: не меньше пяти разделов, есть раздел «чего агент не может знать сам»;
- файл-указатель; всё в git.

Где лежит

- `memory/L1.md`, `L2.md`, `L3.md`;
- `AGENTS.md`, `INDEX.md` в корне проекта.

Идёт в ДЗ 1, срок — к неделе 6.

Итог недели 3

1. Окно ограничено устройством модели: внимание стоит $O(N^2)$ по операциям, KV-кэш растёт по N; облегчённые варианты внимания и смесь экспертов снижают стоимость и окно не расширяют.
2. Внутри окна вес токенов неравен: разбавление внимания, коллапс представлений, потерянная середина; эффективное окно меньше заявленного.
3. Окно агента наполняется выводом инструментов; ввод за сессию растёт как $O(n^2)$.
4. Модель подстраивается под окно: задача задаётся спецификацией и примерами.
5. Сжатие, заметки вне окна, лестница памяти и подагенты выносят лишнее из окна.
6. AGENTS.md — только то, чего агент не может знать сам.

Контекст — управляемая величина: что попадает в окно, решает исследователь.

К неделе 4

Навигационные пометки дают служебный граф проекта: связи придуманы по ходу работы, типы связей не заданы, противоречия не проверяются.

- онтология — явная схема классов, отношений и правил своей темы;
- класс и экземпляр — опасная путаница уровней;
- локальный граф понятий в Obsidian и YAML без тяжёлой инфраструктуры.

Домашняя работа 1 «Среда и контекст»

Что входит — артефакты недель 1–5

- среда в git и цикл с точкой проверки;
- AGENTS.md своего проекта с критерием готовности и разметкой запретов;
- сжатие своей статьи, полный AGENTS.md, навигационные пометки;
- онтология и мини-граф;
- поиск по личной библиотеке и запрос к внешнему графу.

Пакет сдачи

- отчёт: что сделано, как проверено, что не получилось;
- файл `AGENTS.md`;
- примеры запросов к локальному контексту;
- ссылка на репозиторий и метка git.

Сдача к неделе 6, 20 баллов. После срока — половина балла; индивидуальный перенос не рассматривается; дорешка — две недели после курса.

Литература недели 3 I

[26] R. Sennrich, B. Haddow, and A. Birch, "Neural machine translation of rare words with subword units," in *Proc. 54th Annu. Meeting Assoc. Comput. Linguistics (Vol. 1: Long Papers)*, Berlin, Germany, 2016, pp. 1715–1725, doi: 10.18653/v1/P16-1162.

[27] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, "Language models are unsupervised multitask learners," OpenAI, Tech. Rep., 2019.

[28] A. Vaswani *et al.*, "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 30, 2017, pp. 5998–6008, arXiv:1706.03762.

[29] J. Alammar, "The Illustrated Transformer," blog post, Jun. 27, 2018.

[30] J. Alammar, "The Illustrated GPT-2 (Visualizing Transformer Language Models)," blog post, Aug. 12, 2019.

[38] К. В. Воронцов, «Модели внимания и трансформеры», слайды курса «Машинное обучение», МФТИ, 27 февр. 2026. [Онлайн]. Доступно: `http://www.machinelearning.ru/wiki/images/1/19/Voron-ML-Attention-slides.pdf`

[40] A. Yang *et al.*, "Qwen3 technical report," arXiv:2505.09388, May 2025.

[49] Yandex Cloud, "Available generative models," *Yandex AI Studio documentation*.

[70] Anthropic, "Introducing the Model Context Protocol," Nov. 25, 2024; спецификация протокола.

[71] "AGENTS.md — open format for guiding coding agents," Agentic AI Foundation (Linux Foundation).

[72] Anthropic, "Claude Code best practices."

[73] C. E. Shannon, "A mathematical theory of communication," *Bell Syst. Tech. J.*, vol. 27, no. 3, pp. 379–423 and no. 4, pp. 623–656, 1948.

[83] N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. Le, G. Hinton, and J. Dean, "Outrageously large neural networks: The sparsely-gated mixture-of-experts layer," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2017, arXiv:1701.06538.

[85] R. Child, S. Gray, A. Radford, and I. Sutskever, "Generating long sequences with sparse transformers," arXiv:1904.10509, 2019.

[86] N. Shazeer, "Fast transformer decoding: One write-head is all you need," arXiv:1911.02150, 2019.

[87] I. Beltagy, M. E. Peters, and A. Cohan, "Longformer: The long-document transformer," arXiv:2004.05150, 2020.

[88] T. B. Brown *et al.*, "Language models are few-shot learners," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 33, 2020, arXiv:2005.14165.

[89] W. Fedus, B. Zoph, and N. Shazeer, "Switch Transformers: Scaling to trillion parameter models with simple and efficient sparsity," *J. Mach. Learn. Res.*, vol. 23, pp. 1–40, 2022, arXiv:2101.03961.

[90] J. Wu, L. Ouyang, D. M. Ziegler, N. Stiennon, R. Lowe, J. Leike, and P. Christiano, "Recursively summarizing books with human feedback," arXiv:2109.10862, 2021.

[93] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, "FlashAttention: Fast and memory-efficient exact attention with IO-awareness," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 35, 2022, arXiv:2205.14135.

Литература недели 3 II

[94] J. Ainslie, J. Lee-Thorp, M. de Jong, Y. Zemlyanskiy, F. Lebrón, and S. Sanghai, "GQA: Training generalized multi-query transformer models from multi-head checkpoints," in *Proc. EMNLP*, 2023, arXiv:2305.13245.

[96] Z. Zhang *et al.*, "H2O: Heavy-hitter oracle for efficient generative inference of large language models," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 36, 2023, arXiv:2306.14048.

[97] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, "Lost in the middle: How language models use long contexts," *Trans. Assoc. Comput. Linguistics*, vol. 12, pp. 157–173, 2024, arXiv:2307.03172.

[98] B. Peng, J. Quesnelle, H. Fan, and E. Shippole, "YaRN: Efficient context window extension of large language models," in *Proc. ICLR*, 2024, arXiv:2309.00071.

[99] W. Kwon *et al.*, "Efficient memory management for large language model serving with PagedAttention," in *Proc. 29th ACM Symp. Operating Systems Principles (SOSP)*, 2023, doi: 10.1145/3600006.3613165.

[100] G. Delétang *et al.*, "Language modeling is compression," in *Proc. ICLR*, 2024, arXiv:2309.10668.

[101] G. Xiao, Y. Tian, B. Chen, S. Han, and M. Lewis, "Efficient streaming language models with attention sinks," in *Proc. ICLR*, 2024, arXiv:2309.17453.

[102] C. Packer, S. Wooders, K. Lin, V. Fang, S. G. Patil, I. Stoica, and J. E. Gonzalez, "MemGPT: Towards LLMs as operating systems," arXiv:2310.08560, 2023.

[104] A. Q. Jiang *et al.*, "Mixtral of experts," arXiv:2401.04088, 2024.

[105] C.-P. Hsieh *et al.*, "RULER: What's the real context size of your long-context language models?" in *Proc. Conf. Lang. Model. (COLM)*, 2024, arXiv:2404.06654.

[106] DeepSeek-AI, "DeepSeek-V2: A strong, economical, and efficient mixture-of-experts language model," arXiv:2405.04434, 2024.

[107] F. Barbero *et al.*, "Transformers need glasses! Information over-squashing in language tasks," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 37, 2024, arXiv:2406.04267.

[108] J. Howard, "The /llms.txt file," proposal, Sep. 3, 2024.

[109] J. Yuan *et al.*, "Native sparse attention: Hardware-aligned and natively trainable sparse attention," in *Proc. 63rd Annu. Meeting Assoc. Comput. Linguistics (ACL)*, 2025, pp. 23078–23097, arXiv:2502.11089.

[110] K. Hong, A. Troynikov, and J. Huber, "Context rot: How increasing input tokens impacts LLM performance," Chroma Research, Jul. 14, 2025.

[111] Y. Ji, "Context engineering for AI agents: Lessons from building Manus," Manus blog, Jul. 18, 2025.

[112] L. Mei *et al.*, "A survey of context engineering for large language models," arXiv:2507.13334, 2025.

[113] P. Rajasekaran, E. Dixon, C. Ryan, and J. Hadfield, "Effective context engineering for AI agents," Anthropic Engineering Blog, Sep. 29, 2025.

[114] Anthropic, "Equipping agents for the real world with Agent Skills," Anthropic Engineering Blog, Oct. 16, 2025; открытый стандарт с 18 дек. 2025.

[115] The Linux Foundation, "Linux Foundation announces the formation of the Agentic AI Foundation," press release, Dec. 9, 2025.

[116] P. Zeyliger, "Expensively quadratic: The LLM agent cost curve," exe.dev blog, Feb. 3, 2026.

[117] T. Gloaguen, N. Mündler, M. Müller *et al.*, "Evaluating AGENTS.md: Are repository-level context files helpful for coding agents?" arXiv:2602.11988, 2026; ICLR 2026 Workshop on Memory for LLM-Based Agentic Systems.

[118] T. Mason, "The missing memory hierarchy: Demand paging for LLM context windows," arXiv:2603.09023, 2026.

[119] Qwen Team, "Qwen3-235B-A22B-Instruct-2507: model card and configuration," Hugging Face, 2025.

[120] E. Normand, *Grokking Simplicity: Taming Complex Software with Functional Thinking*. Shelter Island, NY, USA: Manning, 2021.

[121] А. П. Халов, О. М. Атаева, «AI4Science: консольный агент курса», репозиторий GitHub, 2026.